

CONSTRAINTCRAFT

SMART CONSTRAINT SOLVING & CONFLICT DETECTION SYSTEM

Programming in Java – Evaluated Project

Submitted by:
Divya Pal

Registration Number: 25BAI10788

B.Tech CSE (AI & ML)
VIT Bhopal University

Academic Year: 2026–27

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope of the Project
5. Target Users
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. System Workflow
10. UML Design
11. Project Modules
12. Java Concepts and Technologies Used
13. Implementation Details
14. Input, Processing and Output
15. Testing and Test Cases
16. Application Results and Screenshots
17. Challenges Faced
18. Learning Outcomes
19. Limitations
20. Future Enhancements
21. Conclusion
22. References

1. INTRODUCTION

ConstraintCraft is a Java-based command-line system designed to solve allocation and scheduling problems using user-defined constraints.

Many real-world problems require assigning entities to available resources while satisfying multiple rules. Examples include arranging students during examinations, allocating classes to rooms, and scheduling tasks into time slots.

ConstraintCraft models these problems using entities, resources and constraints. It validates the input, detects direct contradictory constraints and uses a backtracking-based solution engine to search for a valid assignment.

The project demonstrates object-oriented programming, interfaces, recursion, collections, exception handling, file handling and modular software design.

The system supports three major scenarios: Exam Seating, Room Allocation and Task Scheduling. The application is completely command-line based.

2. PROBLEM STATEMENT

Allocation and scheduling problems often contain multiple restrictions that must be satisfied simultaneously.

In examination seating, two students may not be allowed to sit beside each other while another pair may be required to sit together. In room allocation, classes need to be assigned to available rooms. In task scheduling, tasks may have ordering constraints.

When such problems are handled manually, checking every possible arrangement becomes difficult and time-consuming. Contradictory rules may also remain unnoticed.

ConstraintCraft provides a reusable Java-based constraint solving system. The user enters entities, resources and constraints. The system validates the input, checks for direct conflicts and applies a backtracking algorithm to search for a valid assignment.

3. PROJECT OBJECTIVES

- Develop a Java-based command-line constraint solving system.
- Solve allocation and scheduling problems using backtracking.
- Detect contradictory constraints before or during solving.
- Provide input validation and exception handling.
- Demonstrate object-oriented programming through modular classes.
- Support multiple real-world scenarios using reusable solver architecture.
- Measure solving performance using execution time and backtracking attempts.
- Generate text reports containing solution and performance information.
- Provide a maintainable and extensible project structure.

4. SCOPE OF THE PROJECT

The current version supports three major scenarios.

Exam Seating: Assign students to seats while considering seating constraints.

Room Allocation: Assign classes or subjects to available rooms while considering resource-related constraints.

Task Scheduling: Assign tasks to available time slots while considering ordering constraints.

The current system uses a simplified resource-assignment model. Advanced factors such as room capacity, multiple sessions and optimization preferences are outside the current scope.

5. TARGET USERS

- Educational institutions
- Teachers and administrators
- Scheduling coordinators
- Students learning constraint-solving algorithms
- Users working with basic allocation and scheduling problems

6. FUNCTIONAL REQUIREMENTS

FR1 – Scenario Selection: Select Exam Seating, Room Allocation or Task Scheduling.

FR2 – Entity Input: Enter students, classes or tasks.

FR3 – Resource Input: Enter available seats, rooms or time slots.

FR4 – Constraint Management: Define constraints for the selected problem.

FR5 – Input Validation: Validate user input and handle invalid values.

FR6 – Conflict Detection: Identify directly contradictory constraints where possible.

FR7 – Constraint Solving: Use backtracking to search for a valid assignment.

FR8 – Solution Display: Display a generated assignment or indicate failure.

FR9 – Performance Measurement: Record backtracking attempts and execution time.

FR10 – Report Generation: Generate a text report containing the result and relevant solving information.

7. NON-FUNCTIONAL REQUIREMENTS

NFR1 – Usability: Provide a simple command-line interaction flow.

NFR2 – Performance: Complete small and medium-sized problems within reasonable execution time.

NFR3 – Reliability: Handle invalid input and contradictory constraints without unexpected termination.

NFR4 – Maintainability: Use modular classes and packages.

NFR5 – Extensibility: Allow additional scenarios and constraint types in future.

NFR6 – Portability: Run on systems supporting Java 17 without a graphical environment.

8. SYSTEM ARCHITECTURE

ConstraintCraft follows a modular architecture consisting of model, solver, service, scenario, exception and utility components.

Main Layer: Main.java provides the command-line entry point and coordinates scenarios.

Scenario Layer: ExamSeating, RoomAllocation and TaskScheduling provide specialized workflows.

Service Layer: ConstraintManager manages constraints, ConflictDetector checks conflicts, and ReportGenerator prepares reports.

Solver Layer: ConstraintSolver defines the solving contract and BacktrackingSolver provides the backtracking implementation.

Model Layer: Entity, Constraint, Problem and Solution represent core project data.

Exception Layer: InvalidConstraintException handles invalid constraint-related input.
Utility Layer: FileManager supports file operations used for report generation.

9. SYSTEM WORKFLOW

1. User selects a scenario.
2. User enters entities.
3. User enters available resources.
4. User adds constraints.
5. The system validates input.
6. ConstraintManager stores constraints.
7. ConflictDetector checks for direct conflicts.
8. BacktrackingSolver searches for a valid assignment.
9. The system displays a solution or reports failure.
10. Performance information is recorded.
11. ReportGenerator creates a text report.

Workflow:

User Input → Validation → Constraint Management → Conflict Detection → Backtracking Solver → Valid Solution / Conflict → Performance Measurement → Report Generation

10. UML DESIGN

UML diagrams were created to represent the structure and behavior of ConstraintCraft.

10.1 Use Case Diagram

The Use Case Diagram represents interactions between the user and the system, including Select Scenario, Enter Entities, Add Constraints, Validate Input, Detect Conflicts, Solve Problem, View Solution and Generate Report.

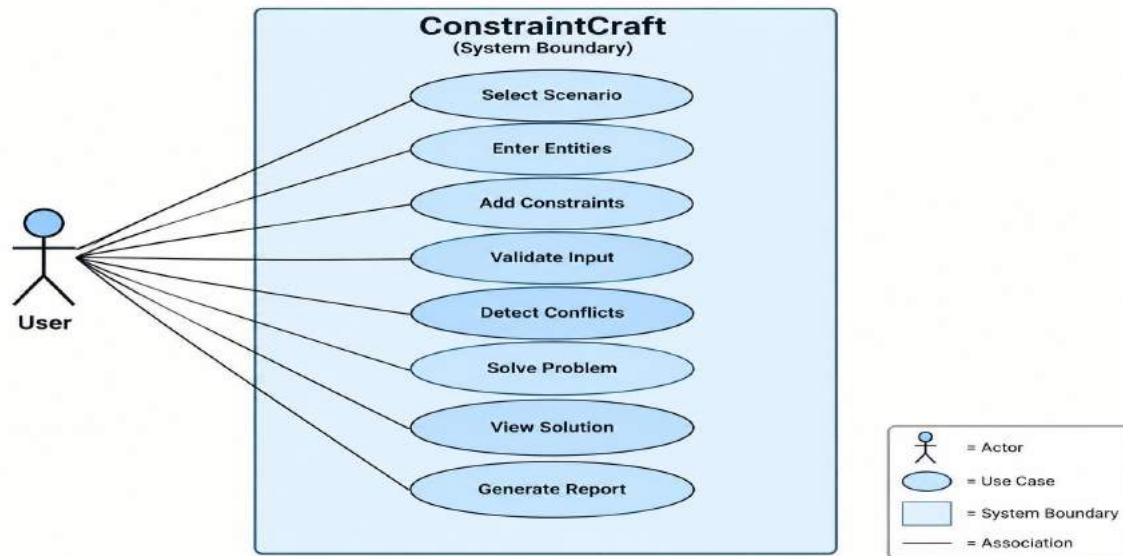


Figure 1: ConstraintCraft Use Case Diagram

10.2 Class Diagram

The Class Diagram represents the 15 major project classes: Main, Entity, Constraint, Problem, Solution, ConstraintSolver, BacktrackingSolver, ConstraintManager, ConflictDetector, ReportGenerator, ExamSeating, RoomAllocation, TaskScheduling, InvalidConstraintException and FileManager.

ConstraintCraft – UML Class Diagram Java 17 Command-Line Project

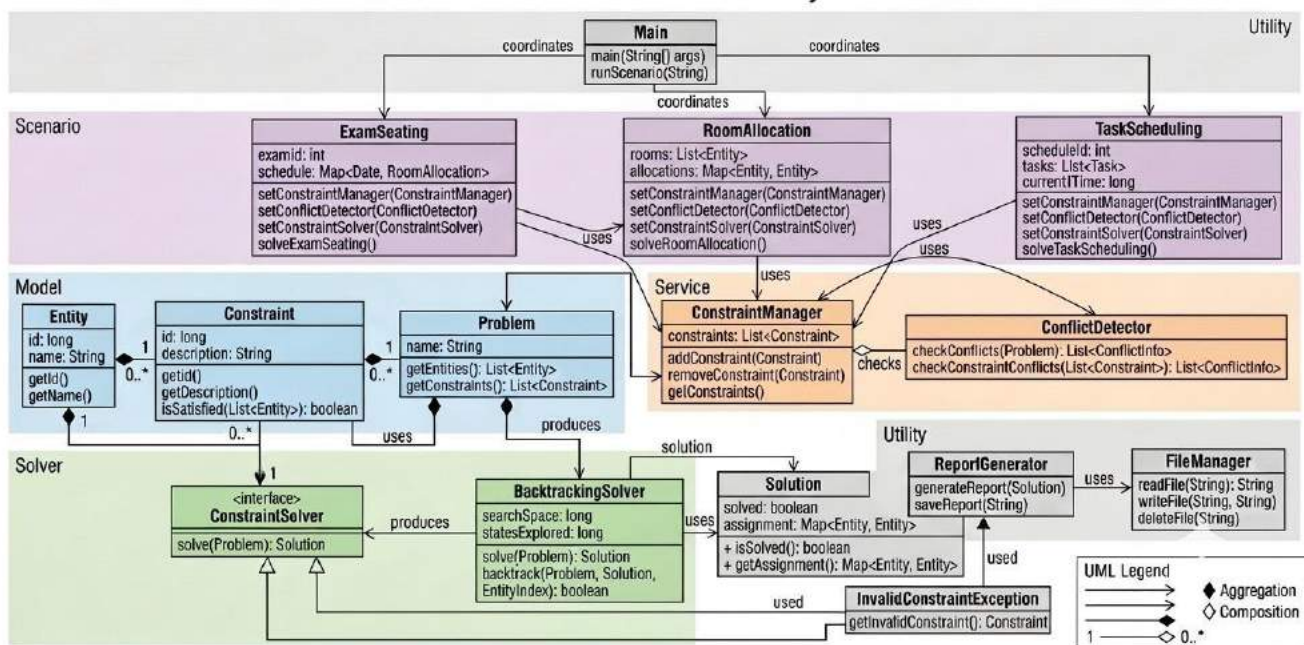


Figure 2: ConstraintCraft Class Diagram

10.3 Component Diagram

The Component Diagram represents the high-level components of the system.

ConstraintCraft – UML Component Diagram

Java 17 Command-Line Constraint Solving System

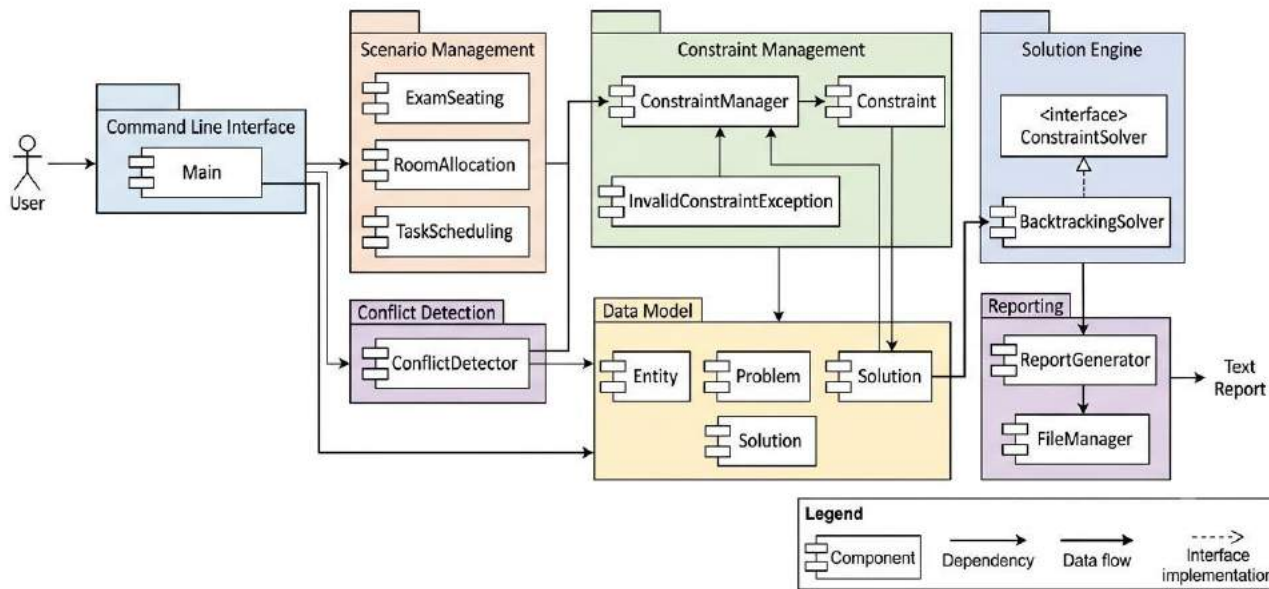


Figure 3: ConstraintCraft Component Diagram

10.4 Sequence Diagram

The Sequence Diagram represents the interaction from scenario selection and constraint entry through conflict detection, solving and report generation.

ConstraintCraft – UML Sequence Diagram

Java 17 Command-Line Constraint Solving System

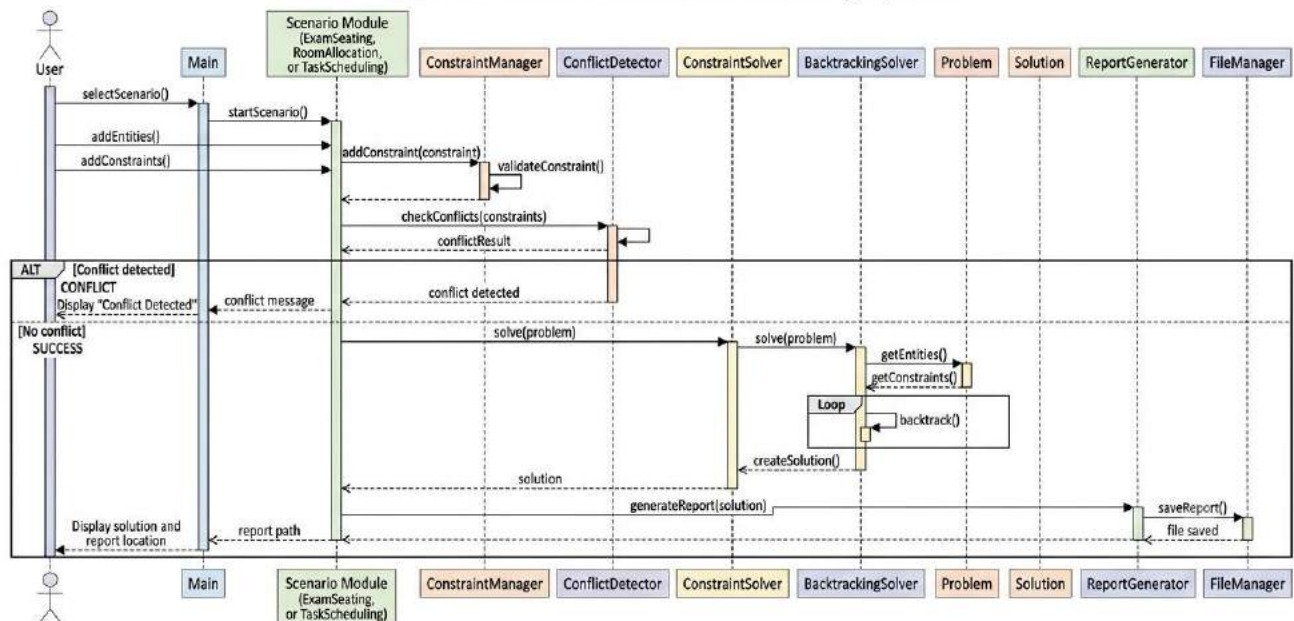


Figure 4: ConstraintCraft Sequence Diagram

11. PROJECT MODULES

11.1 Exam Seating Module

Assigns students to seats while respecting seating constraints.

11.2 Room Allocation Module

Assigns classes or subjects to available rooms.

11.3 Task Scheduling Module

Assigns tasks to available time slots while considering ordering constraints.

11.4 Constraint Management Module

ConstraintManager provides reusable management of user-defined constraints.

11.5 Conflict Detection Module

ConflictDetector checks direct contradictions, such as a pair being both required and forbidden to sit together.

11.6 Solution Engine

BacktrackingSolver incrementally assigns resources, checks constraints and backtracks when an assignment cannot lead to a valid solution.

11.7 Report Generation Module

ReportGenerator creates text reports containing scenario, constraints, solution, attempts and execution time. FileManager handles file writing.

12. JAVA CONCEPTS AND TECHNOLOGIES USED

- Java 17
- Object-Oriented Programming
- Classes and Objects
- Encapsulation
- Interfaces and Abstraction
- Interface Implementation
- Recursion
- Backtracking
- Java Collections
- Exception Handling
- Input Validation
- File Handling
- Modular Programming
- Git and GitHub

13. IMPLEMENTATION DETAILS

The implementation is organized into model, solver, service, scenario, exception and util packages.

Model: Entity.java, Constraint.java, Problem.java, Solution.java

Solver: ConstraintSolver.java, BacktrackingSolver.java

Service: ConstraintManager.java, ConflictDetector.java, ReportGenerator.java

Scenario: ExamSeating.java, RoomAllocation.java, TaskScheduling.java

Exception: InvalidConstraintException.java

Utility: FileManager.java

Entry point: Main.java

Backtracking process:

1. Select the next entity.
2. Select an available resource.
3. Assign the resource temporarily.
4. Check current constraints.
5. Continue recursively if valid.
6. Undo the assignment if it leads to failure.
7. Try another resource.
8. Continue until a solution is found or possibilities are exhausted.

14. INPUT, PROCESSING AND OUTPUT

INPUT

- Scenario selection
- Number of entities
- Entity names
- Available resources
- Constraint types and values

PROCESSING

1. Read input.
2. Validate input.
3. Create the problem model.
4. Store constraints.
5. Check direct conflicts.
6. Execute the backtracking solver.
7. Measure performance.
8. Generate a report.

OUTPUT

- Entered constraints
- Conflict messages when applicable
- Valid assignments
- Backtracking attempts
- Execution time
- Report file location

15. TESTING AND TEST CASES

Test Case 1 – Valid Exam Seating

Input: Four students A, B, C, D; A cannot sit beside B; C must sit beside D.

Expected: Valid seating arrangement.

Actual: Valid arrangement generated.

Status: PASS

Test Case 2 – Conflict Detection

Input: A cannot sit beside B; A must sit beside B.

Expected: Contradictory constraints detected.
Actual: Contradictory rules detected.
Status: PASS

Test Case 3 – Room Allocation

Input: Digital Logic, Java and OS with three rooms.
Expected: Each class assigned to an available room.
Actual: Classes assigned to rooms.
Status: PASS

Test Case 4 – Task Scheduling

Input: Study, Assignment and Revision with three time slots; Study before Assignment; Assignment before Revision.
Expected: Valid schedule.
Actual: Valid schedule generated.
Status: PASS

Test Case 5 – Invalid Input

Input: Non-numeric value where a number is required.
Expected: Error handling and request for valid input.
Actual: Invalid input handled through validation.
Status: PASS

16. APPLICATION RESULTS AND SCREENSHOTS

The following screenshots demonstrate execution of the ConstraintCraft application.

16.1 Exam Seating Result

Four students were tested with seating constraints and a valid arrangement was generated.

Screenshot 1-

```
← → ConstraintCraft

Problems Output Debug Console Terminal Ports

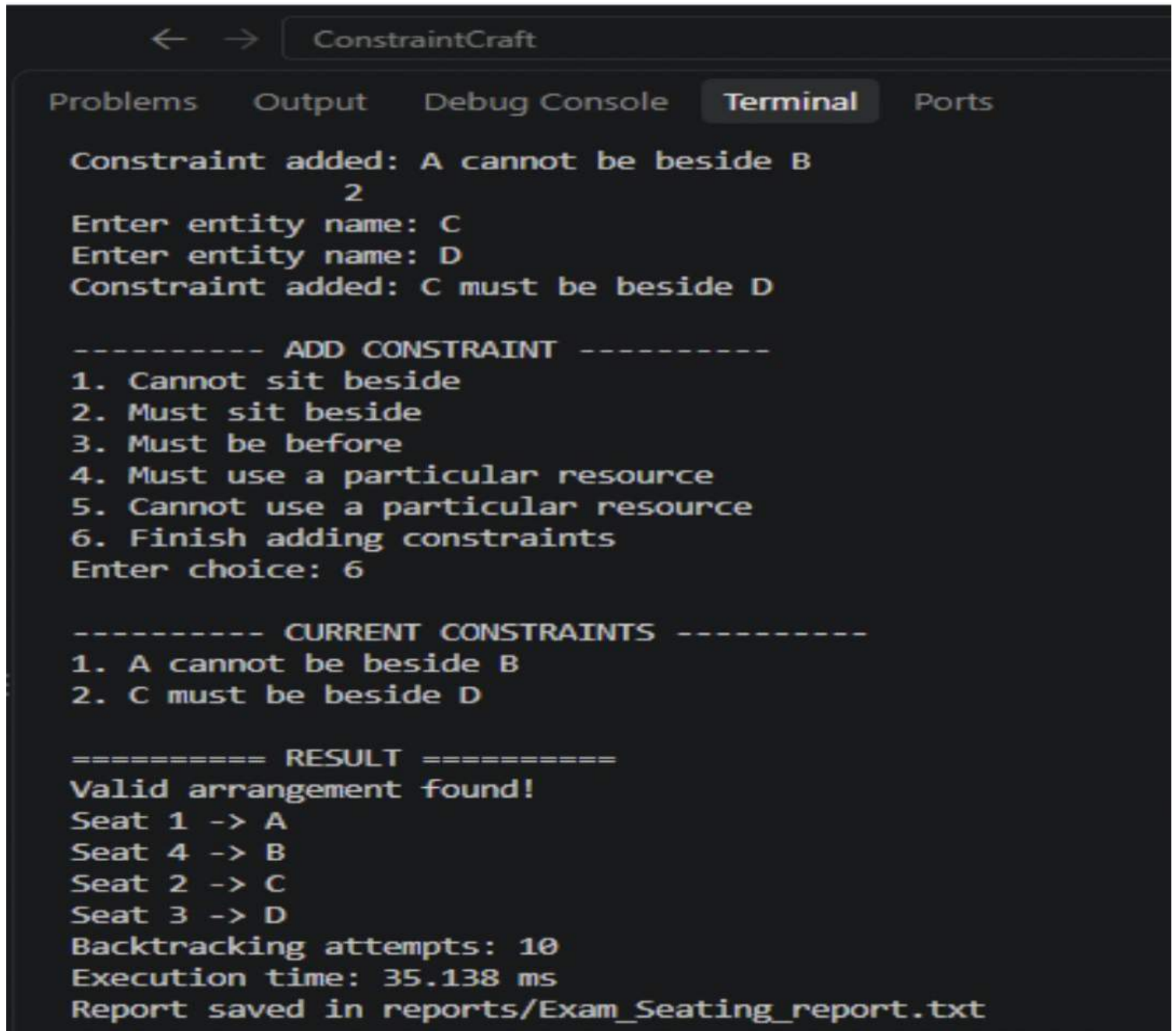
=====
                CONSTRAINTCRAFT
        Smart Constraint Solving System
=====

===== MAIN MENU =====
1. Exam Seating
2. Room Allocation
3. Task Scheduling
4. Exit
Enter your choice: 1

===== EXAM SEATING =====
Enter number of students: 4
Enter student 1 name: A
Enter student 2 name: B
Enter student 3 name: C
Enter student 4 name: D

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 1
Enter entity name: A
Enter entity name: B
Constraint added: A cannot be beside B
                2
Enter entity name: C
Enter entity name: D
Constraint added: C must be beside D
```

Screenshot 2-



```
← → ConstraintCraft

Problems Output Debug Console Terminal Ports

Constraint added: A cannot be beside B
2
Enter entity name: C
Enter entity name: D
Constraint added: C must be beside D

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 6

----- CURRENT CONSTRAINTS -----
1. A cannot be beside B
2. C must be beside D

===== RESULT =====
Valid arrangement found!
Seat 1 -> A
Seat 4 -> B
Seat 2 -> C
Seat 3 -> D
Backtracking attempts: 10
Execution time: 35.138 ms
Report saved in reports/Exam_Seating_report.txt
```

Figure 5: Exam Seating Execution Result

16.2 Room Allocation Result

Three classes and three rooms were tested and room assignments were generated.

Screenshot 1-

```
← → ConstraintCraft
Problems Output Debug Console Terminal Ports

=====
                CONSTRAINTCRAFT
        Smart Constraint Solving System
=====

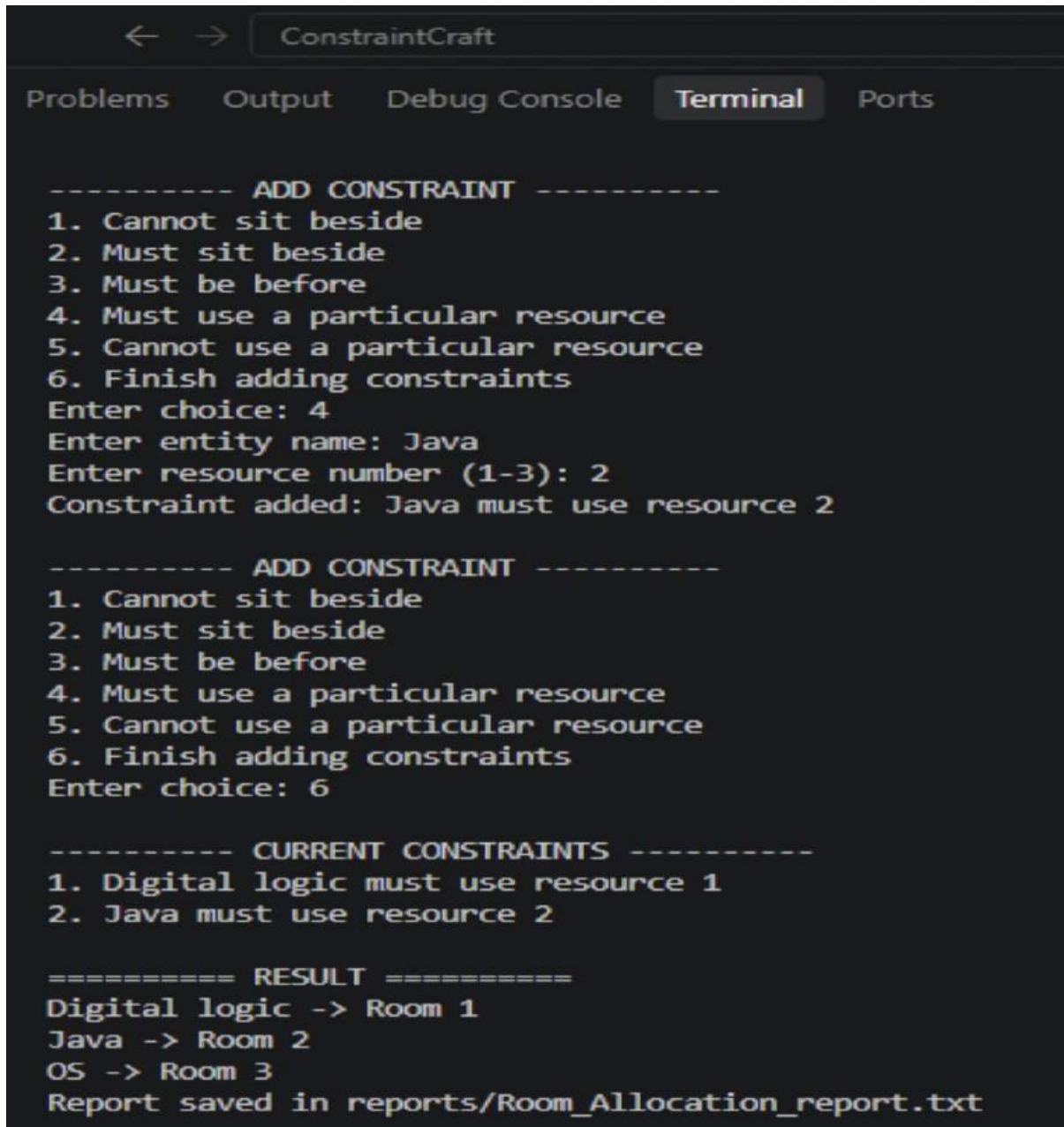
===== MAIN MENU =====
1. Exam Seating
2. Room Allocation
3. Task Scheduling
4. Exit
Enter your choice: 2

===== ROOM ALLOCATION =====
Enter number of classes: 3
Enter number of rooms: 3
Enter class 1 name: Digital logic
Enter class 2 name: Java
Enter class 3 name: OS

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 4
Enter entity name: Digital logic
Enter resource number (1-3): Room 1
Please enter a valid number.
Enter resource number (1-3): 1
Constraint added: Digital logic must use resource 1

----- ADD CONSTRAINT -----
1. Cannot sit beside
```

Screenshot 2-



```
← → ConstraintCraft
Problems Output Debug Console Terminal Ports

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 4
Enter entity name: Java
Enter resource number (1-3): 2
Constraint added: Java must use resource 2

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 6

----- CURRENT CONSTRAINTS -----
1. Digital logic must use resource 1
2. Java must use resource 2

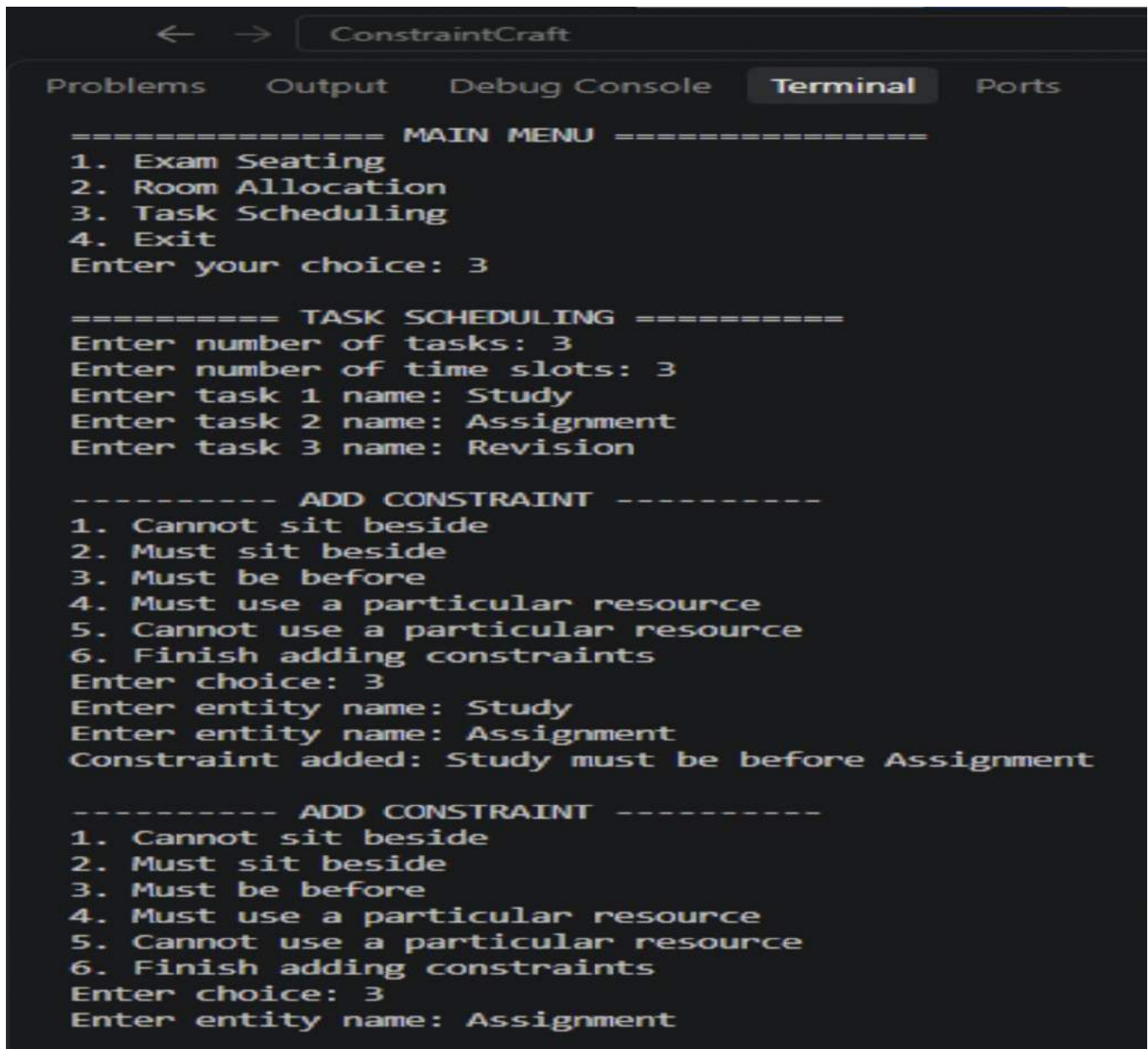
===== RESULT =====
Digital logic -> Room 1
Java -> Room 2
OS -> Room 3
Report saved in reports/Room_Allocation_report.txt
```

Figure 6: Room Allocation Execution Result

16.3 Task Scheduling Result

Three tasks and three time slots were tested with ordering constraints and a valid schedule was generated.

Screenshot 1-



```
← → ConstraintCraft
Problems Output Debug Console Terminal Ports

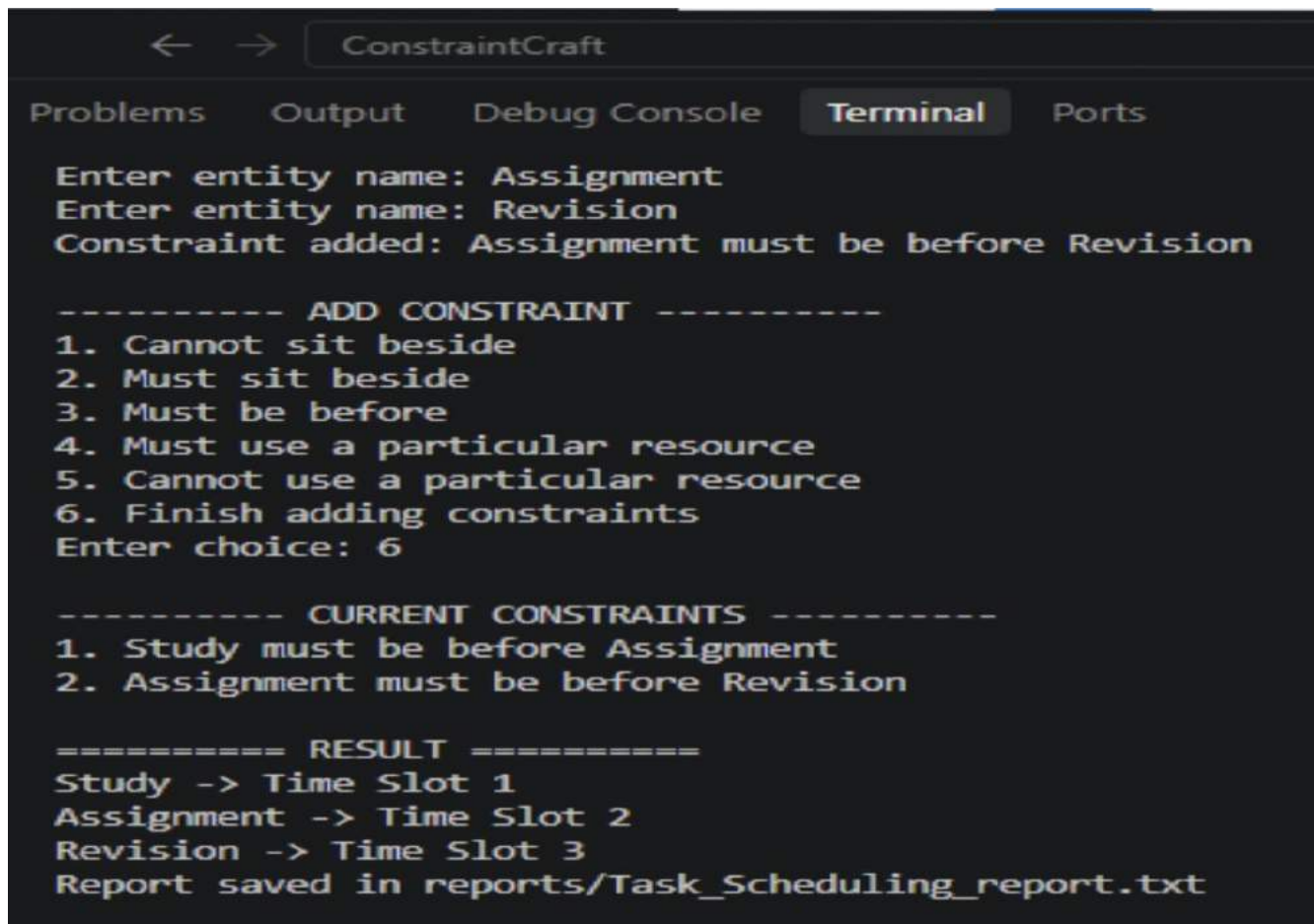
===== MAIN MENU =====
1. Exam Seating
2. Room Allocation
3. Task Scheduling
4. Exit
Enter your choice: 3

===== TASK SCHEDULING =====
Enter number of tasks: 3
Enter number of time slots: 3
Enter task 1 name: Study
Enter task 2 name: Assignment
Enter task 3 name: Revision

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 3
Enter entity name: Study
Enter entity name: Assignment
Constraint added: Study must be before Assignment

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 3
Enter entity name: Assignment
```

Screenshot 2-



```
← → ConstraintCraft
Problems Output Debug Console Terminal Ports

Enter entity name: Assignment
Enter entity name: Revision
Constraint added: Assignment must be before Revision

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 6

----- CURRENT CONSTRAINTS -----
1. Study must be before Assignment
2. Assignment must be before Revision

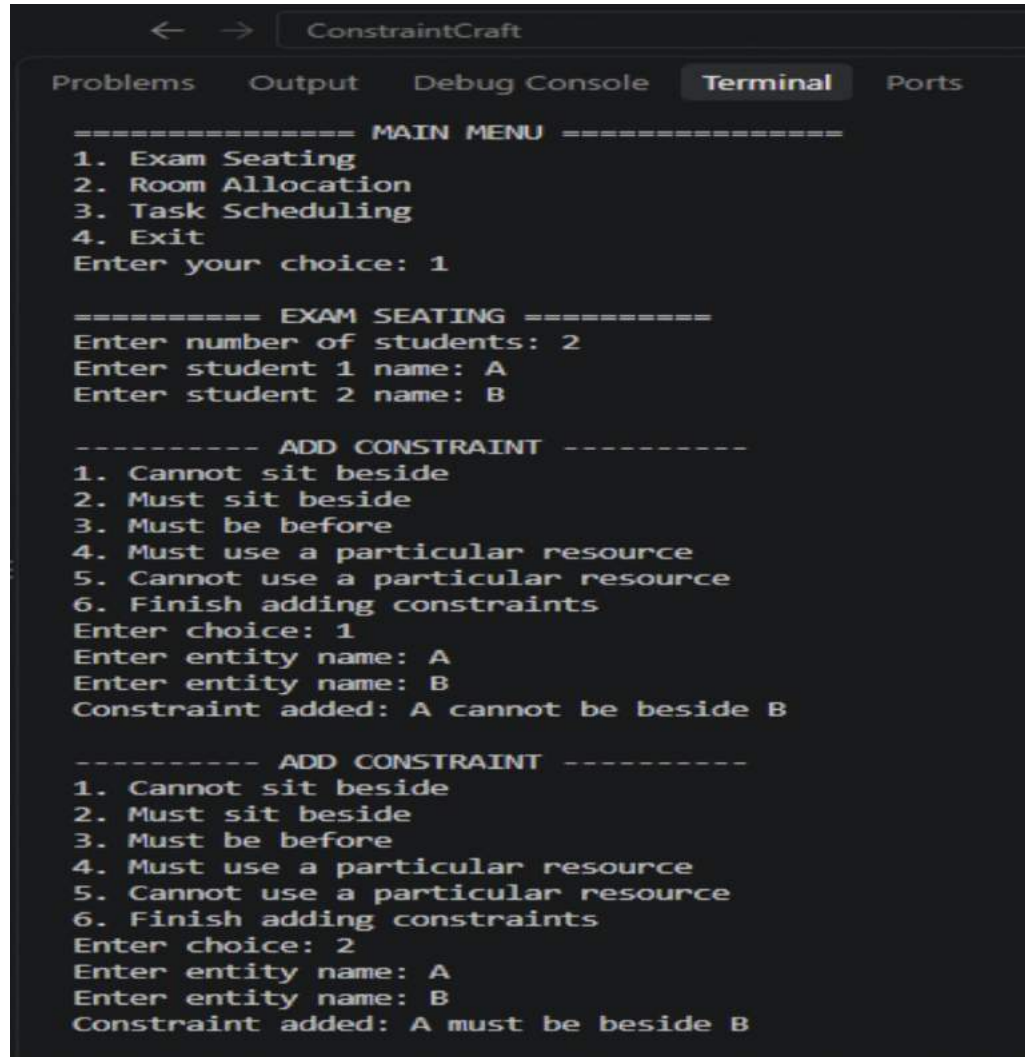
===== RESULT =====
Study -> Time Slot 1
Assignment -> Time Slot 2
Revision -> Time Slot 3
Report saved in reports/Task_Scheduling_report.txt
```

Figure 7: Task Scheduling Execution Result

16.4 Conflict Detection Result

Contradictory rules were tested and the system detected the conflict.

Screenshot 1-



```
← → ConstraintCraft
Problems Output Debug Console Terminal Ports

===== MAIN MENU =====
1. Exam Seating
2. Room Allocation
3. Task Scheduling
4. Exit
Enter your choice: 1

===== EXAM SEATING =====
Enter number of students: 2
Enter student 1 name: A
Enter student 2 name: B

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 1
Enter entity name: A
Enter entity name: B
Constraint added: A cannot be beside B

----- ADD CONSTRAINT -----
1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints
Enter choice: 2
Enter entity name: A
Enter entity name: B
Constraint added: A must be beside B
```

Constraint added: A must be beside B

----- ADD CONSTRAINT -----

1. Cannot sit beside
2. Must sit beside
3. Must be before
4. Must use a particular resource
5. Cannot use a particular resource
6. Finish adding constraints

Enter choice: 6

----- CURRENT CONSTRAINTS -----

1. A cannot be beside B
2. A must be beside B

CONFLICT DETECTED

- Contradictory rules for A and B.

Figure 8: Conflict Detection Execution Result

17. CHALLENGES FACED

- Designing a reusable constraint model for different scenarios.
- Implementing recursive backtracking correctly.
- Detecting contradictory constraints.
- Handling invalid command-line input.
- Organizing the project into modular packages.
- Generating and storing solution reports.

18. LEARNING OUTCOMES

The project provided practical experience with object-oriented programming, classes and objects, interfaces and abstraction, recursion, backtracking, Java Collections, exception handling, input validation, file handling, modular programming, software architecture, UML design, testing and debugging, and Git/GitHub version control.

19. LIMITATIONS

- Each entity is assigned to a unique resource in the current simplified model.
- Advanced resource capacity is not currently implemented.
- Multiple sessions for the same resource are not modeled.
- The solver focuses on finding a valid solution rather than optimizing it.
- Only a limited set of constraint types is currently supported.
- Reports are file-based rather than database-backed.

20. FUTURE ENHANCEMENTS

1. Resource Capacity Management – Consider room capacities and student counts.
2. Advanced Constraint Types – Add availability, priority and exclusion rules.
3. Scheduling Optimization – Compare valid solutions using optimization criteria.
4. Larger Datasets – Improve solver efficiency for larger problems.
5. Database Integration – Store users, problems, constraints and solutions.
6. Additional Scenarios – Support employee scheduling, event planning and examination timetables.
7. Improved User Interface – Add an advanced interface while preserving the core Java architecture.

21. CONCLUSION

ConstraintCraft demonstrates how Java programming concepts and algorithmic problem solving can be combined to develop a practical constraint-based system.

The project provides three scenarios: Exam Seating, Room Allocation and Task Scheduling. It allows users to enter constraints, validates input, detects direct conflicts and uses a backtracking algorithm to search for valid assignments.

The modular architecture separates models, services, scenarios, solver logic, exceptions and utilities. This improves maintainability and makes the system easier to extend.

The project successfully demonstrates practical use of object-oriented programming, recursion, backtracking, collections, exception handling and file handling in Java.

22. REFERENCES

1. Oracle Java Documentation – Java Platform Documentation.
2. Oracle Java Documentation – Collections Framework.
3. Oracle Java Documentation – Exception Handling.
4. Oracle Java Documentation – File and I/O APIs.
5. Java Programming course material and lecture resources.
6. General algorithmic reference material on recursion and backtracking.
7. GitHub Documentation – Git and repository management.